

Intel SGX Enclave Hardware Security Cheat-sheet

L0 Essence:

[M1.1] Processor Reserved Memory (PRM) & EPC Allocation

\\textbf{P}: During BIOS initialization, the PRMRR register sets aside a contiguous segment of RAM called Processor Reserved Memory (PRM) that is inaccessible to any off-CPU devices (including DMA). The Enclave Page Cache (EPC) resides inside the PRM to hold secure enclaves. \\textbf{D}: EPC pages are allocated via the SGX kernel driver. In Linux 5.11+, the core allocator function is `sgx_alloc_epc_page`, distributing 4KB physical pages. PRM size is bios-configured and limited; exceeding it causes paging. \\textbf{S}: `main.c:L30-80`

[M1.2] EPCM Hard Access Check

\\textbf{P}: EPCM (Enclave Page Cache Map) is an internal hardware table within the CPU storing safety metadata for each EPC page (permissions R/W/X, page type SECS/REG/TCS, and owner SECS ID). \\textbf{D}: Even if the OS maps an EPC physical page to an untrusted process, the MMU checks the EPCM and triggers a General Protection fault (`#GP`) if the processor is not executing inside the matching Enclave context. \\textbf{S}: `encl.c:L120-160`

[M1.3] Memory Encryption Engine (MEE)

\\textbf{P}: The MEE is embedded in the memory controller. It encrypts cache lines written to PRM via AES-XTS and decrypts them on read, shielding data from hardware probes. \\textbf{D}: To prevent replay and relocation attacks, MEE builds a Memory Cryptographic Tree (MCTS) with its root hash stored in CPU registers, turning bus communications into pseudorandom noise. \\textbf{S}: `main.c:L180-210`

[M1.4] SGX Lifecycle Instructions (ECREATE/EADD/EINIT/EEXTEND)

\\textbf{P}: Enclave setup sequence: ECREATE initializes SECS metadata; EADD copies pages into the EPC; EEXTEND cryptographically hashes 256-byte chunks of pages; EINIT seals the enclave measurement registers. \\textbf{D}: EINIT requires a cryptographically signed SIGSTRUCT file containing a valid Launch Token. Enclave initialization fails if platform launch keys are mismatched. \\textbf{S}: `ioctl.c:L200-290`

[M1.5] Enclave Execution Limit

\\textbf{P}: Once in Enclave mode, the CPU operates in a restricted User Mode (Ring 3), and most privileged system instructions (e.g., IN, OUT, INT, CLI, STI) are hardware-blocked. \\textbf{D}: Any attempts to change privilege rings or handle I/O within the enclave force an AEX, preserving the security integrity of the enclave and the host OS. \\textbf{S}: `encl.c:L300-340`

[M2.1] AEX State Cleanup

\\textbf{P}: When interrupts/exceptions occur inside the enclave, CPU prevents register leakage to untrusted handlers by executing an Asynchronous Enclave Exit (AEX), clearing all general-purpose registers. \\textbf{D}: The hardware saves the active CPU register values onto the enclave's State Save Area (SSA) frame. Control resumes via ERESUME, restoring the SSA states back. \\textbf{S}: `assembly.S:L10-60`

[M2.2] Page Fault Side-Channel

\\textbf{P}: The host OS retains page table control. A malicious OS can intentionally trigger selective page faults (`#PF`) to trace internal control flow based on memory access patterns. \\textbf{D}: Mitigations involve software techniques such as Oblivious RAM (ORAM) or utilizing hardware timers to detect page fault cycle frequency anomalies. \\textbf{S}: `encl.c:L410-450`

[M2.3] L1TF / Foreshadow Defence

\\textbf{P}: Foreshadow exploits CPU speculative execution. Even when page tables mark data as non-present, out-of-order execution loads EPC cache lines from L1D Cache. \\textbf{D}: Mitigations include disabling Hyper-Threading (HT), flushing L1 Data Cache on enclave transitions, and restricting core pin mappings in microcode. \\textbf{S}: `main.c:L310-350`

[M2.4] TCS & SSA Struct

\\textbf{P}: TCS (Thread Control Structure) is a hardware-reserved page specifying thread entry points (OENTRY) and SSA offsets. Each active CPU thread inside the enclave requires a distinct TCS. \\textbf{D}: The SSA holds space for saving general-purpose registers, floating-points, and SSE states. These pages must be added via EADD before calling EINIT. \\textbf{S}: `encl.h:L50-80`

[M2.5] EPC Page Swapping

\\textbf{P}: If EPC memory gets exhausted, the OS can swap enclave pages to normal RAM. This requires executing EBLOCK to block accesses, and ETRACK to clear TLB cache. \\textbf{D}: The driver invokes EWB to encrypt and copy the target page into standard system memory, along with a version token. ELDU reloads and decrypts the page. \\textbf{S}: `ioctl.c:L340-410`

L1 Logical Framework

L2 Data Flow Topology

Trade-off Matrix: Security vs Performance

[M3.1] Local Attestation

\\textbf{P}: Intra-platform enclaves establish mutual trust using EREPORT, which generates a report cryptographically bound to the target's SECS measurement via a platform-specific symmetric key. \\textbf{D}: The target enclave calls EGETKEY to derive the same Report Key and verifies the MAC checksum, ensuring both run on the same genuine SGX processor. \\textbf{S}: `attestation/local.c:L15-60`

[M3.2] Remote Attestation Quote

\\textbf{P}: Remote attestation wraps an enclave report into an exportable certificate (Quote). The report is handed over to the Quoting Enclave (QE), a local architectural enclave. \\textbf{D}: QE validates the local report, then signs it using a platform-specific private key issued by the PCE, transforming the report into an off-platform Quote. \\textbf{S}: `attestation/quote.c:L40-95`

[M3.3] Intel PCS Certificate

\\textbf{P}: The verifying party needs to prove that the QE Quote public key originates from a genuine Intel CPU. Intel PCS publishes PCK certificate CRLs online. \\textbf{D}: Verifiers fetch the platform's PCK (Processor Certification Key) cert chain signed by Intel CA to verify the hardware trust path back to Intel's factory. \\textbf{S}: attestation/pcs.c:L10-45

[M3.4] EPID vs ECDSA Signatures

\\textbf{P}: Early SGX Remote Attestation relied on Intel EPID (Enhanced Privacy ID) zero-knowledge group signatures. Modern SGX uses standard ECDSA signatures for local verification. \\textbf{D}: ECDSA quotes are signed by QE3 and validated using local cached certificate servers (PCCS), eliminating internet latencies to Intel's cloud. \\textbf{S}: attestation/ecdsa.c:L50-110

[M3.5] MRENCLAVE & MRSIGNER

\\textbf{P}: MRENCLAVE is a SHA-256 hash measuring the enclave's code, data, and load order ("what it is"). MRSIGNER is the hash of the developer's public signing key ("who signed it"). \\textbf{D}: Enclave updates change MRENCLAVE but preserve MRSIGNER. This allows developers to use MRSIGNER policy to share sealed database keys across versions. \\textbf{S}: sdk/tlibc/tattest.c:L20-55

[M4.1] ECALL/OCALL Context Switch

\\textbf{P}: ECALL transfers control inside the enclave via EENTER/EEXIT. OCALL makes callbacks to host functions. Each transition incurs heavy registers/state flush overhead. \\textbf{D}: Context switching costs several thousand CPU cycles. Large-scale enclaves employ "Switchless Enclaves" utilizing shared lockless ring buffers to prevent exit. \\textbf{S}: edger8r/codegen.ml:L40-80

[M4.2] Parameter Marshalling

\\textbf{P}: Enclave code must not directly dereference memory pointers passed from the outside (vulnerable to TOCTOU attacks). Enclave bridge code must copy arguments securely. \\textbf{D}: Auto-generated Edger8r boundary wrappers verify whether pointers lie strictly outside the enclave boundary, copy content into EPC heap, then perform operations. \\textbf{S}: edger8r/parser.ml:L100-140

[M4.3] SGX2 EDMM Dynamic Memory

\\textbf{P}: SGX1 required enclave EPC sizes to be statically defined at compilation. SGX2 introduces EDMM (Enclave Dynamic Memory Management) hardware support. \\textbf{D}: EDMM allows runtime page allocation (EMODT) and permission changes (EMODPR), enabling dynamic heap allocation via standard malloc routines. \\textbf{S}: sdk/trts/trts_util.cpp:L30-70

[M4.4] Runtime LibOS Systems

\\textbf{P}: Since enclaves cannot trigger system calls directly, running unmodified binaries (like Redis or Nginx) requires a Library OS (e.g. Gramine/Occlum) compatibility layer. \\textbf{D}: The LibOS intercepts standard system calls and simulates them inside the enclave (such as secure file systems or network loops), shielding the host OS from raw calls. \\textbf{S}: sdk/tlibc/string/memset.c:L5-20

[M5.1] Debug vs Prod Mode

\\textbf{P}: Debug mode enclaves configure the DEBUG flag inside SECS, allowing the CPU to execute EDGRD and EDWR instructions to inspect EPC pages. \\textbf{D}: Production enclaves set DEBUG to 0. Hardware debug registers and JTAG debuggers are physically locked out, returning null or causing hard crash. \\textbf{S}: sdk/simulation/tcs.cpp:L40-75

[M5.2] XML Configurations

\\textbf{P}: Enclave parameters (StackSize, HeapSize, TCSNum) are defined in a custom XML configuration file loaded during compilation. \\textbf{D}: Attributes are parsed by `sgx_sign` and packed into the SIGSTRUCT metadata, matching the MRENCLAVE measurement upon ECREATE register checks. \\textbf{S}: sdk/sign_tool/SignTool/SignTool.cpp:L120-170

[M5.3] `sgx_sign` & SIGSTRUCT

\\textbf{P}: `sgx_sign` generates the SIGSTRUCT file containing measurements, attributes, and public key of the developer, signed with their RSA-3072 private key. \\textbf{D}: The hardware instruction EINIT validates the signature before executing code. This prevents malicious modification of EPC code files. \\textbf{S}: sdk/sign_tool/SignTool/sgx_sign.cpp:L200-280

[M5.4] Simulation Mode Boundaries

\\textbf{P}: Simulation Mode runs on non-SGX hardware by mimicking EPCM structures and AEX flows in software. \\textbf{D}: MEE encryption is missing in simulation mode, and local attestation reports rely on simulated software keys. Must never be used in production. \\textbf{S}: sdk/simulation/assembly/

[M5.5] Architectural Enclaves

\\textbf{P}: Architectural Enclaves (AEs) are system enclaves developed by Intel (LE, QE, PCE) to manage platform-specific tasks. \\textbf{D}: AEs possess keys signed by Intel Root. They can access hardware-specific platform keys that normal user enclaves are restricted from reading. \\textbf{S}: psw/ae/common/

[M6.1] EPC Paging Tuning

\\textbf{P}: Paging thrashing happens when an enclave allocates memory exceeding physical EPC limits, triggering heavy encryption/decryption overhead. \\textbf{D}: Tuning involves leveraging NUMA-aware layouts, reducing global scans, adopting cache-friendly data structures, and optimizing XML initial allocation sizes. \\textbf{S}: arch/x86/kernel/cpu/sgx/main.c#L450-510

[M6.2] Multi-Enclave IPC

\\textbf{P}: Enclaves are memory-isolated from each other. They communicate by establishing shared symmetric keys or using non-EPC shared RAM pages. \\textbf{D}: Enclaves exchange encrypted messages via non-EPC ring buffers. The message is decrypted only inside the target enclave, avoiding OS sniffing. \\textbf{S}: sdk/attestation/local.c#L80-120

[M6.3] KVM vEPC Virtualization

\\textbf{P}: KVM supports virtual EPC (vEPC) by directly mapping physical EPC segments into guest VM memory spaces. \\textbf{D}: Hypervisors trap guest VM ECREATE and EINIT operations, updating EPT page tables to handle virtual EPC translations seamlessly. \\textbf{S}: kvm/vmx/sgx.c:L20-100

[M6.4] SVN & TCB Recovery

\\textbf{P}: `ISVSVN` denotes enclave versions. Microcode updates increment physical CPUSVN to recover from architectural security vulnerabilities. \\textbf{D}: Following TCB recovery, PCS revokes old PCK certificates. Enclaves compare SVN and CPUSVN, refusing to start if host lacks vital security patches. \\textbf{S}: sdk/tlibc/tattest.h:L10-40 \\textbf{S}:

Zone T1: Intel SGX Core APIs

`sgx_create_enclave()`

`sgx_destroy_enclave()`

`sgx_ecall() / sgx_ocall()`

`ECREATE / EADD / EINIT / EEXTEND`

`EREPORT / EGETKEY`

Zone T2: Errors & Security Protection

`SGX_ERROR_OUT_OF_EPC` (EPC OOM Page Eviction)

`SGX_ERROR_INVALID_SIGSTRUCT`

`SGX_ERROR_ENCLAVE_LOST` (Power transition / crash)

Asynchronous Enclave Exit (AEX)

Controlled Channel Defence (ORAM / Timers)